

Task 04 — Secure API Gateway

Implement these **4 requirements** in your Spring Boot API Gateway:

1. JWT validation
2. Authentication filter
3. Unauthorized response — 401
4. Forbidden response — 403

Step 1 — Add dependencies

In the **API Gateway pom.xml**, add:

```
<dependencies>

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-jose</artifactId>
</dependency>

</dependencies>
```

If spring-cloud-starter-gateway is already present in your project, don't add it again.

Step 2 — Configure JWT

Create/update:

Src/main/resources/application.yml

server: port: 8080

spring: application: name: api-gateway

security: oauth2: resourceserver: jwt: secret-key: my-secret-key-123456789012345678901234567890

cloud: gateway: routes: - id: user-service uri: http://localhost:8081 predicates: - Path=/users/**

- id: order-service

uri: http://localhost:8082

predicates:

- Path=/orders/**

Use the **same JWT secret** that your authentication/JWT service uses.

Step 3 — Create Security Configuration

Create:

SecurityConfig.java

```
package com.example.gateway.config;
```

```
import org.springframework.context.annotation.Bean; import  
org.springframework.context.annotation.Configuration; import  
org.springframework.security.config.annotation.web.reactive.EnableWebFluxSecurity;  
import org.springframework.security.config.web.server.ServerHttpSecurity; import  
org.springframework.security.web.server.SecurityWebFilterChain;
```

```

@Configuration @EnableWebFluxSecurity public class SecurityConfig {

@Bean
public SecurityWebFilterChain securityWebFilterChain(
    ServerHttpSecurity http) {

return http
    .csrf(ServerHttpSecurity.CsrfSpec::disable)

    .authorizeExchange(exchange -> exchange

        // Public API
        .pathMatchers("/auth/**").permitAll()

        // Admin API
        .pathMatchers("/admin/**")
        .hasRole("ADMIN")

        // Manager API
        .pathMatchers("/manager/**")
        .hasAnyRole("ADMIN", "MANAGER")

        // User API
        .pathMatchers("/users/**")
        .hasAnyRole("USER", "MANAGER", "ADMIN")

        // Everything else requires authentication
        .anyExchange().authenticated()
    )

    .oauth2ResourceServer(oauth2 ->
        oauth2.jwt(jwt -> {})
    )

    .exceptionHandling(exception -> exception

        // 401 - no/invalid authentication
        .authenticationEntryPoint(

```

```

        (exchange, ex) -> {
            exchange.getResponse()
                .setStatusCode(
                    org.springframework.http.HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        })

        // 403 - authenticated but insufficient role
        .accessDeniedHandler(
            (exchange, ex) -> {
                exchange.getResponse()
                    .setStatusCode(
                        org.springframework.http.HttpStatus.FORBIDDEN);
                return exchange.getResponse().setComplete();
            })
        )

        .build();
    }
}

```

Step 4 — JWT Authentication Filter

Create:

JwtAuthenticationFilter.java

```
package com.example.gateway.filter;
```

```
import org.springframework.cloud.gateway.filter.GatewayFilter;
```

```
import
```

```
org.springframework.cloud.gateway.filter.factory.AbstractGatewayFilterFactory;
```

```
import org.springframework.http.HttpHeaders;
```

```
import org.springframework.stereotype.Component;
```

```
@Component
```

```
public class JwtAuthenticationFilter
```

```
    extends AbstractGatewayFilterFactory<Object> {
```

```

public JwtAuthenticationFilter() {
    super(Object.class);
}

@Override
public GatewayFilter apply(Object config) {

    return (exchange, chain) -> {

        String authHeader = exchange.getRequest()
            .getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null ||
            !authHeader.startsWith("Bearer ")) {

            exchange.getResponse().setStatusCode(
                org.springframework.http.HttpStatus.UNAUTHORIZED);

            return exchange.getResponse().setComplete();
        }

        return chain.filter(exchange);
    };
}
}

```

This filter checks whether the request contains:

Authorization: Bearer <JWT_TOKEN>

Step 5 — Add the filter to Gateway routes

Update application.yml:

spring: cloud: gateway: routes:

- id: user-service
 - uri: http://localhost:8081
 - predicates:
 - Path=/users/**
 - filters:
 - JwtAuthenticationFilter

- id: order-service
 - uri: http://localhost:8082
 - predicates:
 - Path=/orders/**
 - filters:
 - JwtAuthenticationFilter

Now the request flow is:

Client

|

| Authorization: Bearer JWT

↓

API Gateway

|

|-- JWT missing → 401 Unauthorized

|

|-- JWT invalid → 401 Unauthorized

|

|-- JWT valid

↓

Check Role

|

|-- Wrong role → 403 Forbidden

|

|-- Correct role

↓

Microservice

Expected responses

No token:

HTTP/1.1 401 Unauthorized

Invalid/expired token:

HTTP/1.1 401 Unauthorized

Valid token but wrong role:

HTTP/1.1 403 Forbidden

Valid token + correct role:

HTTP/1.1 200 OK

Task 04 is Completed.

We have implemented,

Requirement	Implementation
JWT validation	OAuth2 Resource Server
Authentication filter	JwtAuthenticationFilter
Unauthorized response	401 Unauthorized
Forbidden response	403 Forbidden
Role-based access	hasRole() / hasAnyRole()

